ICS 214 Lab 7

Intro

The focus in this lab is coding the Gameboard class which develops functionality to manipulate, set, and query the state of the gameboard grid on which blocks are placed. We will make use of the Point class, so Point.cpp and Point .h will need to be fully developed/tested before beginning this lab.

Objectives:

1. Introduce working with 2D arrays
2. Write a class to encapsulate the functionality of the Gameboard
3. Observe how breaking down tasks into functions with a singular purpose, can give us the tools we need to develop functionality with more complex behaviours.
4. Familiarization with vectors & ranged for loops.
5. Using asserts to check for valid parameters

The Gameboard Class



The Gameboard class encapsulates the functionality of the Tetris game board. The gameboard class simply manages a 2D array of content (integers). Each integer can represent either an empty spot, or a colored block.

Our gameboard class doesn't need to know anything about Tetrominos. This allows us to keep the Gameboard as self-contained as possible and have it focus on dealing with its own content (blocks that have been placed on a Gameboard). We will be writing Gameboard methods to operate on that content (particularly on rows) to facilitate operations that typically happen on a Tetris gameboard (checking for a complete row, filling a row, copying a row, collapsing rows, etc…).

Actual gameplay (the moving of a Tetromino) will be handled by a separate TetrisGame class (in a future lab) which will control both a GridTetromino and the gameboard. A GridTetromino inherits from Tetromino. It adds the functionality of being aware of its XY location (on a gameboard). It will eventually reach a location on the gameboard where it cannot move down any further (because the spaces that it would occupy already contain content). At this point, the information that the Tetromino contains about its blocks (location and color) will be copied to the gameboard grid. We use the gameboard to determine what points are already occupied as a result of previous Tetromino blocks being "copied" onto the grid.

The Gameboard will represent empty spots as -1's, and areas that are occupied by blocks of color (content) as integers. These will have been copied onto the board from past Tetrominos that have already been placed.

Representing the gameboard in code:

- The game board is represented by a 2D array of integers (the grid).
- The array contains content (integers) which represent either:
  - an EMPTY_BLOCK (-1),
  - An int representing color from the Tetromino::TetColor enum.
- This 2D array is oriented with [0][0] at the top left and [MAX_Y-1][MAX_X-1] at the bottom right. Why? This makes it far easier to draw the gameboard on the screen since it matches with the way screen pixels co-ordinates work (pixel 0,0 is considered top left).
  - A 2D array uses row-major ordering by default – which means that the first index is the row and the second is the column. eg: grid[row][col]. However, it is going to be easier for us to think of the gameboard grid in terms of XY coordinates. Here's the catch… **X represents columns and Y represents rows**. I realize this seems counter-intuitive. To make sense of it - think of pixels on a screen, or XY co-ordinates on a co-ordinate system. Does a change in X change the row or the column? The column right? Similarly a change in Y affects the row.
  - **This means that when we index into a 2D array with an x, y co-ordinate, we are going to need to reverse what we would probably do naturally: (grid[x][y]), and index the grid the other way around (grid[y][x]). This reversal is only needed internal to the gameboard class (since grid is a private member variable). Clients don't need to reverse X and Y. When clients call public Gameboard methods (eg: setContent(x,y, content)) they can treat X and Y as we normally would.** This is an example of **abstraction** - the details of the implementation are hidden from the client.
- As you develop the code, add assert statements to the tops of functions to ensure parameters are what you expect them to be. For example: if I write setContent(x, y, content) it will help to add an assert statement that will halt the program when x or y is out of range. Without an assert, passing an invalid index to X or Y would result in undefined behavior since you will be reading/writing off the end of an array - which can be a nightmare to debug! These assert statements will help you to detect errors early.

Developing the gameboard: the plan for implementation:

Creating simple, reusable, singular purpose methods

For me, one of the most fun things about game programming – is looking at a complex problem, and then trying to figure out how to break it into smaller sub-problems that are more manageable. A big



part of this challenge is also developing objects and methods in a way that makes code as re-usable and elegant as possible. I tend to think of code as "elegant" if it is simple to understand, and it is built in a way that allows it to be as re-usable as possible (usually facilitated by creating singular purpose methods). This allows us to build up a "toolbox" of objects and methods that we create for ourselves which will make the complexities of coding easier. A hammer and a wrench are tools that get used a lot because they have a singular purpose. This allows them to be re-used for a lot of different applications. This approach to programming can be applied to pretty much any language you're working in – not just C++.

If you haven't played Tetris at this point (what??), or if you could use a refresher on how the game works – go to this link (https://tetris.com/play-tetris) and play for a couple minutes. Observe the behavior of the Gameboard (specifically) so that you get a sense of what functionality we need to build. Notice things like:

- What happens when a Tetromino gets placed onto a gameboard?
- What happens to a row when it is completely filled?
- What effect does a row completion have on all the blocks in the rows above it?

Once you've thought about how the gameboard works, take a moment to consider what types of "bite sized", single purpose, reusable methods we might want the Gameboard class to have (and how we can implement them). For example it would be very helpful to have a method to:

- determine if a row is completed – so we know when to collapse a row.
- copy the contents of the row above it into itself (for collapsing rows)
- fill a row with a given value (for filling the top row with empty blocks, or for initializing the whole gameboard to empty values).

Gameboard constructor, setters and getters:
To start with, we'll want a constructor to initialize the gameboard to empty. We'll also want some setters and getters to allow a client to set and retrieve the "content" at XY locations on the gameboard. It may be convenient to pass these setters/getters XY values, or a Point, so we'll accommodate both.

Future work on the Tetromino to support Gameboard functionality:T
Let's think ahead for a moment. Eventually (in upcoming labs) we're going to want to be able to move a Tetromino around on a gameboard. A Tetromino is aware of the locations of its own blocks in relation to its center (Point(0,0)), but it knows nothing of a Gameboard. If we were to add functionality (eventually) to a Tetromino to give it an awareness of its XY offset on a gameboard, we could then write a Tetromino method to return a vector of the XY points to represent its block locations relative to that offset (on the Gameboard). In other words, if a Tetromino knows that it is located at co-ordinate 5,5 (on the Gameboard), this new Tetromino function could return a vector of points where it translates Point(0,0) to Point(5,5), along with the other point offsets for the other blocks it contains. That information is going to be super

handy – especially if we write some gameboard functions to make use of it! We'll look at writing this new Tetromino method in a future lab, but for now – let's assume it's doable.

Tetromino Movement:
In Tetris gameplay, a Tetromino's movement is restricted to empty spots on the gameboard. To determine where you can legally move a Tetromino, we can make a temporary copy of the Tetromino, move the copy to the requested location, get the copy's block XY offsets on the gameboard, and then check if those locations on the gameboard contain empty spots. If the locations are all empty, it is a valid move. To do this, let's write a Gameboard method called areAllLocsEmpty() to check if a vector of XY offsets (Points) are all empty. We can later use this to test if a Tetromino can be legally moved to a certain location.

Placing a Tetromino:
When you "place" a Tetris block, what actually happens? The Tetromino blocks (represented by color enums) are copied onto the gameboard. The Tetromino can then be reused by repositioning it at the top of the Gameboard and setting it to a different shape. To support copying a Tetromino to the Gameboard, let's write another Gameboard method called setContent() to take in a vector of a Tetromino XY offsets and an int to represent content (color) so that we can request the point offsets from a Tetromino, and then set the gameboard contents with the information we receive.

Removing a row:
Now that we've thought about how we might want to move and place a Tetromino on a Gameboard, let's turn our attention to how a Gameboard works in the world of Tetris. When a row is completely filled on a Gameboard, what happens? The Gameboard moves all the rows above the completed row down 1 row. The completed row gets overwritten by the content (the colors or spaces) in the row above it. This keeps happening all the way up the gameboard until row 0 is copied into row 1. At that point row 0 is filled with empty blocks. That should take care of collapsing a completed row.

Let's break this somewhat complex functionality down into some simpler Gameboard methods to make it easier to understand and tackle. We'll first need a method to detect if a row is completely filled with content (no empty blocks). That should be easy. We'll call it isRowCompleted().

It would be helpful to know what row indices have been completed. We can then write a function called getCompletedRowIndices() to go through each row in the entire gameboard, call isRowCompleted() on each row index. If the row is complete, add the row index to a vector of row indices. If we return this vector we will have a list of all the completed row indices so we know which rows need to be removed.

Once we have a vector of completed row indices, we can turn our attention to writing a function that will deal with removing a row: removeRow(rowIndex). Since removing a row is really about stepping backwards (bottom to top) row-by-row, and continually overwriting a

row's contents with the contents of the row above it, let's write a function that can be re-used for this: copyRowIntoRow(sourceRowIndex, targetRowIndex). To implement the functionality of removeRow(), we'll start at the row we want to remove (y), call copyRowIntoRow(y-1, y) and then work our way back to row 1 in a for loop. Once we've copied row 0 into row 1, we just to fill row 0 with empty blocks. We can write another method to do this last step called: fillRow(rowIndex, content).

Removing multiple rows:

In theory, we now have a function to remove (and collapse) a row, and we have another function that provides us with a vector of row indices to remove. We should be able to write a function removeRows() that takes a vector of row indices to remove as a parameter, and iterates through the vector calling removeRow(rowIndex) on each index. In this way, we can collapse all the completed rows with a single function call! Beautiful!



Phew! I think that should do it. That's probably a lot to take in, but I wanted to give an overview of the thought process behind writing the gameboard class so that you have an idea of **why** you're writing the methods you are writing, and how they all fit together to give us the functionality we need for behaviour we see in the Gameboard when playing Tetris.

When I'm breaking a somewhat complex task into smaller tasks like this, I'm constantly thinking about how I can write the smaller methods to be as reusable as possible. Generally, if I can break tasks down into sub-tasks that have a singular purpose, it is more likely it is that I will be able to reuse those subtasks. The more code I reuse, the less I need to write (and debug). Win-win! The DRY principle really comes into play here. If you find yourself repeating the same code - chances are good that you can parcel the logic into a smaller singular-purpose functions that can be reused.

Have a look at the UML diagram on the next page for an overview of the functionality breakdown of the Gameboard class.

The Gameboard Class

Member functions exists for:

- accessing (getting/setting) the grid content.
- testing if rows are completed (all content non empty)
- testing if specific locations are empty
- filling rows with the same content,
- emptying the board
- removing a row (which handles cascading rows above it downward)
- copying rows into other rows
- providing a spawn location (to represent a point where tetrominoes will start from)

[expected .cpp size: ~ 225 lines]

Gameboard

+ const int MAX_X = 10
+ const int MAX_Y = 19
+ const int EMPTY_BLOCK = -1
- int grid[MAX_X][MAX_Y];
- const Point spawnLoc {MAX_X/2, 0}

+ Gameboard()
+ void empty()
+ void printToConsole()
+ int getContent(Point pt)
+ int getContent(int x, int y)
+ void setContent(Point pt, int content)
+ void setContent(int x, int y, int content)
+ void setContent(vector<Point> locs, int content)
+ bool areAllLocsEmpty(vector<Point> locs)
+ int removeCompletedRows()
+ Point getSpawnLoc()
- bool isValidPoint(Point pt)
- bool isValidPoint(int x, int y)
- bool isRowCompleted(int rowIndex)
- void fillRow(int rowIndex, int content)
- vector<int> getCompletedRowIndices()
- void copyRowIntoRow(int sourceRow, int targetRow)
- void removeRow(int rowIndex)
- void removeRows(vector<int> rowIndices)

The parameters in this UML diagram are currently ALL shown as pass by value. As you implement these methods, determine when it is appropriate to use:

- **pass by value,**
- **pass by reference**
- **pass by const reference**

… and modify the parameters accordingly.

The Gameboard.h header file has been provided on D2L for this lab. It contains essentially the same info that the UML diagram contains – along with comments on each individual function and hints for developing it. The hints give you tips on how the functions can be reused, but also on specific implementation details.

Estimated size of Gameboard.cpp is ~150 lines of code (which accounts for 25% of the Tetris project. When this lab is complete, the Tetris project will be approximately 50% done.

Getting Started

- Create a new empty project called Lab7.
- Create a Main.cpp file and write a main function as you would normally. We will use this for testing later.
- Download 3 files from the Lab7 folder on D2L.
    - TestSuite.h
    - TestSuite.cpp
    - Gameboard.h
- Copy these files into your project folder.
    - Right click on the Lab7 project name in the Solution Explorer (if not visible, select View | Solution Explorer).
    - Select 'Open folder in File Explorer' and paste them to the folder that opens.
    - Right click on Solution Explorer | Lab7 | Header Files, select Add | Existing Item, and then select the header files you just copied.
- Copy your completed version of Point.h and Point.cpp from the previous lab into the same folder and add it to the project using the method described above.
- In your solution explorer (in VS2022) – drag the .h files into the "Header files" folder.
- Create Gameboard.cpp (Right click on the Source files folder & Add | New Item).
- Read through all the documentation at the top of Gameboard.h.
- Fill in the function signatures in the header file based on the documentation.
    - Use good (meaningful) variable names.
    - Use best practices for determining whether to pass by value, reference, or const reference (see table in Chapter 9 class slides, appendix B if unsure).
    - Label any methods that should be const as const.
- Copy all the function declarations (and comments) into the Gameboard.cpp file.
    - Add "Gameboard::" before all method names in your .cpp to indicate the methods belong to the Gameboard class.
    - Add braces ({}) to each method to get things compiling.
    - Use temporary return values for methods where needed.

We're ready to begin coding! Methods in the header file are grouped by public/private accessors. Because of that, the ordering does not reflect the order in which you would logically implement the methods. To make logical sense, follow the order listed below.

Coding (and testing) Gameboard functionality

Begin adding the Gameboard functionality (in Gameboard.cpp) in the following order:.

- **fillRow()**
- **empty()**
- **constructor()**
- **printToConsole()**
- **isValidPoint() methods**
- **getContent() methods**
- **setContent() methods**
- **isRowCompleted()**

Test your newly written methods to ensure they work (see below for an example). You will need to make main() a friend (temporarily) in order to access/test private methods.

```
56  class Gameboard {
57      friend int main();  // allow main() to access private vars (for testing)
```

Main.cpp

Lab7 (Global Scope)

```
1   #include <iostream>
2   #include "Gameboard.h"
3   #include "TestSuite.h"
4
5   int main() {
6       Gameboard board;
7       Point pt1{ 1,1 };
8       Point pt2{ 2,2 };
9
10      board.setContent(pt1, 1);
11      board.setContent(pt2, 2);
12      board.fillRow(3, 3);
13      board.printToConsole();
14
15      std::cout << board.getContent(pt1);    // expected: 1
16      std::cout << board.getContent(3,3);    // expected: 3
17      std::cout << board.isRowCompleted(1); // expected: 0
18      std::cout << board.isRowCompleted(3); // expected: 1
19  }
```

Microsoft Visual Studio Debug Console

```
. . . . . .
. 1 . . . . .
. . 2 . . . .
3 3 3 3 3 3 3 3 3 3
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .

1301
```

Continue by implementing the methods used for row manipulation

- **copyRowIntoRow()**
- **removeRow()**
- **removeRows()**
- **getCompletedRowIndices()**

Test these new methods in main() to ensure they work (see below for an example)

Main.cpp — Lab7 — (Global Scope)

```cpp
#include <iostream>
#include "Gameboard.h"
#include "TestSuite.h"

int main() {
    Gameboard board;
    // fill the board with indices matching the rows
    for (int i = 0; i < Gameboard::MAX_Y; i++) {
        board.fillRow(i, i);
    }
    board.removeRow(18);    // remove the lowest row
    board.removeRow(18);    // remove the lowest row
    board.printToConsole(); // examine the results

    // print the completed row indices
    std::vector<int> rowIndices = board.getCompletedRowIndices();
    for (int i : rowIndices) {
        std::cout << i << " ";
    }
}
```

Microsoft Visual Studio Debug Console

```
. . . . . .
. . . . . .
0 0 0 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5
6 6 6 6 6 6 6 6 6 6
7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9 9
10101010101010101010
11111111111111111111
12121212121212121212
13131313131313131313
14141414141414141414
15151515151515151515
16161616161616161616

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
```

Finish off with these last two methods and test to ensure they work.

- **removeCompletedRows()**
- **areAllLocsEmpty()**

```cpp
int main() {
    Gameboard board;
    // fill the board with indices matching the rows
    for (int i = 0; i < Gameboard::MAX_Y; i++) {
        board.fillRow(i, i);
    }
    // make rows 3,6,9 incomplete by setting col 0 to empty
    board.setContent(0,3, Gameboard::EMPTY_BLOCK);
    board.setContent(0,6, Gameboard::EMPTY_BLOCK);
    board.setContent(0,9, Gameboard::EMPTY_BLOCK);

    // view before and after removing completed rows
    board.printToConsole();
    board.removeCompletedRows();
    board.printToConsole();

    // test invalid and empty/occupied points
    std::vector<Point> emptyLocs{ Point(-1,-1), Point(0, 0) };
    std::cout << board.areAllLocsEmpty(emptyLocs);     // expected:1
    std::vector<Point> occupiedLocs{ Point(100,100), Point(1,18) };
    std::cout << board.areAllLocsEmpty(occupiedLocs);  // expected:0
}
```

Microsoft Visual Studio Debug Console

```
0 0 0 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2
. 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5
. 6 6 6 6 6 6 6 6 6
7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 8 8 8 8 8
. 9 9 9 9 9 9 9 9 9
10101010101010101010
11111111111111111111
12121212121212121212
13131313131313131313
14141414141414141414
15151515151515151515
16161616161616161616
17171717171717171717
18181818181818181818

. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. 3 3 3 3 3 3 3 3 3
. 6 6 6 6 6 6 6 6 6
. 9 9 9 9 9 9 9 9 9

10
```

Automated Testing

Once you have completed the Gameboard class:

Put the following statements above your main() function:

```
#include "TestSuite.h"
```

Add a call to the TestSuite in main():

```
// run some sanity tests on our classes to ensure they're working as expected.
TestSuite::runTestSuite();
```

Open the header file of the class you are testing (Gameboard.h) and add this line inside the class definition to allow TestSuite access to private variables & functions of the class:

```
// FRIENDS
friend class TestSuite;// (allows TestSuite access to private members for testing)
```

Uncomment the following lines TestSuite.h:



Run this to prove to yourself that all your Gameboard member functions work. If they don't – look at the test that TestSuite.h is halting at. Try to track down the errors and fix them. Your code will need to pass this test for the demo.

Note: passing TestSuite.h does not guarantee that everything works perfectly. Some errors are difficult to automate testing for. Take care to ensure that your functions do exactly what they are supposed to do. If they do not – problems may surface in the next lab and will need to be fixed there. TestSuite does not do a good job of testing if your setter methods ignore XY points out of range. Ensure these only operate on valid XY points (to save yourself hours of debugging caused by undefined behaviour)

Demo

Show me your code running and passing the TestSuite

Marking: [out of 10]

Marking will be done via a demo, a code walkthrough, and verification that TestSuite runs without any issues.

- Does it work as expected?
- Variables: Have good variable & const names been used?
- Are variables in local scope instead of global?
- Are there comments to explain anything non-trivial or not obvious? Is there a comment at the top of every function?
- Have variables been initialized with best practices (uniform initialization).
- Have variables or constants been used instead of “magic numbers” in the code.
- Have asserts been added to functions to check for valid parameters?